

Créer des class permettra de changer les paramètres plus tard

Pour la camera

```
import { PerspectiveCamera } from "three";

export default class Camera extends PerspectiveCamera {

  constructor() {

    super (70, innerWidth/innerHeight)

    this.position.set(0,0,4)

    this.lookAt(0,0,0)

  }

}
```

La camera est une class qui donnera un point initial et dans app.js quand on créer l'instance (l'objet) on pourra passer des paramètres permettant de jouer avec la camera.

le cas du projecteur de lumière

```
import { AmbientLight, PointLight, Object3D } from "three";

export default class Light extends Object3D {

  constructor() {

    super()

    const ambient = new AmbientLight(0xffffff, 0.7)

    const point = new PointLight(0xffffff)

    point.position.set(1,0,4)

    this.add(ambient)

    this.add(point)

  }

}
```

```
}  
}
```

Ici c'est défini comme un objet 3D et ce projecteur on lui inscrit une couleur et intensité par ambient et son départ et couleur par point.

Pour le moteur graphique

```
import { WebGLRenderer, Clock } from "three";  
  
export default class Graphic extends WebGLRenderer {  
  
  // Les paramètres passés à la classe  
  scene = null  
  clock = new Clock()  
  camera = null  
  
  // Callback appelé à chaque frame (si défini avec onUpdate)  
  cbUpdate = null  
  
  // Fonction de boucle liée au bon contexte  
  cbLoop = null  
  
  constructor(scene, camera) {  
  
    // Initialise le WebGLRenderer et le lie au canvas HTML  
    super({ canvas })  
  
    // 1. Stocke la scène  
    this.scene = scene  
  
    // 2. Stocke la caméra  
    this.camera = camera  
  
    // Lie la méthode loop() à l'instance pour requestAnimationFrame  
    this.cbLoop = this.loop.bind(this)  
  
    // Active le rendu des ombres (la bonne syntaxe serait shadowMap.enabled)  
    this.shadowMap.enable = true  
  
    // Démarre immédiatement la boucle de rendu  
    this.loop()  
  }  
}
```

```
// Boucle d'animation principale
loop() {

    // Récupère le temps écoulé depuis la frame précédente
    const dt = this.clock.getDelta()

    // Si une fonction de mise à jour est fournie, on l'exécute
    if (this.cbUpdate) this.cbUpdate(dt)

    // Dessine la scène avec la caméra
    this.render(this.scene, this.camera)

    // Demande à relancer une frame
    requestAnimationFrame(this.cbLoop)
}

// Permet d'enregistrer une fonction appelée à chaque frame (update loop)
onUpdate(callback) {
    this.cbUpdate = callback
}
}
```